PROJECT ABSTRACT

Student Codex

A student-focused academic companion built for Partido State University — one Android app and one teacher dashboard that unify deadlines, guidance, and community on a single secure backend.

StudentCodex Project · Partido State University

OCTOBER 24, 2026 · VERSION 1.0

Table of Contents

1. Introduction: What StudentCodex Is 1

2. The Problem We Are Solving 1

3. Why It Matters: The Impact 3

4. The Solution, Feature by Feature 4

4.1 Accounts and onboarding 4

4.2 Home: the workload dashboard 4

4.3 The Activity Beacon 4

4.4 The floating Guidance Assistant 5

4.5 The Student Community 5

4.6 Profile and Settings 5

4.7 The teacher web dashboard 5

4.8 Google Classroom readiness 6

5. Architecture and Technology 7

6. Security and Privacy by Design 8

7. Current Status and Known Limitations 8

8. Conclusion and Outlook 9

1. Introduction: What StudentCodex Is

StudentCodex is a student-focused academic companion: a native Android application, built with Kotlin and Jetpack Compose, paired with a teacher web dashboard, both running on one shared Supabase backend. The system was designed around the real academic structure of Partido State University (ParSU) in Camarines Sur, Philippines - its seven campuses and the official registry of forty-two program-campus combinations - so that every account, class, and deadline is anchored to authentic institutional data rather than free-text fields.

The application serves two audiences with one source of truth. Students receive three connected modules: the Activity Beacon, which consolidates every deadline and task a student is responsible for; the Guidance Assistant, a floating helper that is always one tap away on any screen; and the Student Community, a moderated, institution-scoped space for questions and shared knowledge. Teachers receive a dashboard that aggregates cohort workload per section, previews candidate deadlines before they are published, and triages student-reported activities. Because both sides read the same PostgreSQL database, what a student sees and what a teacher plans from can never drift apart.

The name is deliberate. A codex is a single bound volume that replaces scattered scrolls - and that is precisely the product thesis. Where a student's academic life today is scattered across Google Classroom, messenger groups, whiteboards, and memory, StudentCodex binds it into one place: one workload list, one assistant, one community, and one honest picture of the week ahead.

2	3	7	42
platforms: Android app and teacher web dashboard	student modules on one backend	ParSU campuses in the seeded registry	official program-campus combinations

2. The Problem We Are Solving

The core question that motivated this project is simple: where does a student look to know what is due tomorrow? In the status quo, the honest answer is "everywhere, partially." Coursework lives in Google Classroom, but only per course, and only for the courses that happen to use it. Face-to-face requirements are announced on whiteboards, in class group chats, and verbally - none of which reconcile with each other. Personal study plans and organizational tasks live in yet another to-do app, if anywhere at all. The result is a fragmented picture that the student must mentally merge every single day.

> *"Where does a student look to know what is due tomorrow? In the status quo, the honest answer is: everywhere, partially."*

2.1 Fragmented deadlines and duplicated effort

Because each source shows only its own slice, the same assignment can appear twice - once as a Classroom item and once as a hand-copied reminder - while a different one is missed entirely. There is no cross-source identity model that says "these two records are the same task." Without deduplication, workload counts are wrong in both directions, and students either double-submit or discover deadlines after they have passed.

2.2 No honest view of the week ahead

Even when deadlines are visible, no tool answers the question that actually matters: how heavy is my week? A calendar shows events, a to-do app shows an undifferentiated list, and Classroom shows per-course work with no aggregate. Students cannot see that four submissions land on the same Friday until they are already inside the crunch. Planning backwards from a clustered deadline becomes impossible, so work quality and sleep both degrade in exactly the weeks that matter most.

2.3 Teachers schedule blind

The same opacity runs in the other direction. An instructor publishing a new deadline has no practical way to see what is already due for that section that week, across all of the section's other subjects. Deadline pile-ups are usually nobody's fault and everybody's problem: each teacher optimizes locally, and the student absorbs the sum. A tool that shows the section's aggregated workload before a deadline is published changes the conversation from reactive complaints to proactive scheduling.

2.4 Asking for help has a social cost

Academic questions go where they scroll away fastest: group chats and comment threads with no structure, no history, and no moderation. Reserved students often will not ask at all when asking means broadcasting their confusion to classmates. There is no persistent, searchable, institution-scoped space where a question can be asked - optionally without a name attached - and answered for everyone's benefit.

2.5 Tools that overpromise erode trust

Finally, students have learned to distrust polished apps whose content silently fails: spinners that never resolve into data, empty screens with no explanation, generated answers that sound authoritative and are wrong. A tool that wants to be believed must be honest about its states - loading, empty, offline, failed - and must label the provenance of everything it shows. This project treats honest interface states and

source verification as first-class features, not polish.

3. Why It Matters: The Impact

StudentCodex concentrates its impact where the fragmentation hurts most. For students, the Activity Beacon becomes the single place a deadline lives: imported from Google Classroom when that integration is enabled, entered manually for face-to-face and personal work, and deduplicated by a source-identity model so nothing is counted twice. The Home dashboard turns that list into an honest weekly picture - deadlines grouped by day, a due-soon count, and provenance labels that say whether an item is teacher-confirmed, imported, or student-reported. Fewer missed deadlines and earlier awareness of crunch weeks are the direct, measurable outcomes.

The Guidance Assistant lowers the cost of asking. It floats over every screen as a draggable chat head or an edge panel, so help is one tap away without leaving the current task. Its rule-based engine deliberately answers only what it actually knows - study technique, time management, app navigation - and says so plainly when it does not know, because a helper that fabricates university policy is worse than no helper at all. The Student Community completes the support loop: questions can be asked anonymously, answered in threads, liked, and reported for moderation - a persistent knowledge base scoped to the institution instead of a scrolling group chat.

For teachers, the dashboard converts goodwill into information. Before publishing a deadline, an instructor can check the section’s aggregated workload for the target week, compare two candidate dates, and read a plain-language recommendation. Student-reported face-to-face activities arrive as pending evidence that the teacher confirms or rejects - confirmed items become teacher-confirmed activities visible to the whole section. The reporting loop that once ran through group chats becomes an auditable workflow.

For the institution, the impact is structural. Onboarding is driven by the official ParSU registry - campus first, then only the programs that campus actually offers - so institutional data stays aligned with reality. Every table is protected by row-level security, anonymous posting is anonymous at the database layer rather than by convention, and a moderation queue receives content reports. The school gets a digital commons with the same boundaries and accountability as the physical one.

117	11	24	3
automated unit tests in the Android project	versioned database migrations, all applied live	row-level-security policy test scenarios	ways to sign in: email, Google, or demo guest

4. The Solution, Feature by Feature

This chapter walks through the shipped product as a user meets it. Each feature described here exists in the codebase today, is covered by automated tests where testable, and was verified end-to-end on a real device before being accepted. The companion volume, the Development Chronicle and Roadmap, records how each of these features came to be.

4.1 Accounts and onboarding

The app opens on an authentication gate with three ways in. Email sign-up collects first and last name, derives a unique username from them, and then asks for school details - campus first, then the programs that campus actually offers, then year level and section - all from database-backed dropdowns seeded with the official ParSU registry. Google Sign-In uses the Android Credential Manager and exchanges a Google ID token through Supabase Auth; the first Google sign-in completes the same name and school-details form, and adds a password so the account also works with manual login. Duplicate emails and Google accounts are blocked server-side. A guest bypass ("Explore without an account") opens the full app on demo data for evaluation, and sessions persist across app restarts.

4.2 Home: the workload dashboard

Home answers the week’s question at a glance. Upcoming deadlines are grouped by date in flat, dense rows, with overdue work emphasized in the error color and a computed summary line stating exactly how many deadlines fall in the next seven days. Every item carries a course and source label, and the screen handles its empty, loading (skeleton), and error states honestly instead of pretending. An "Ask a study question" affordance expands the floating assistant directly from Home, so the student never has to navigate away to get help.

4.3 The Activity Beacon

The Beacon is the student’s complete activity list: filterable, grouped by date, with trust metadata rendered as plain text - source, verification status, and effort where it is known. Manual activities are created, edited, and deleted through a single full-width entry point. Face-to-face activities that a student records are personal until a teacher confirms them; the reporting pipeline files them as pending evidence for teacher triage rather than silently inserting them as class workload, which keeps the Beacon truthful for everyone.

4.4 The floating Guidance Assistant

The assistant abandoned the conventional "chat tab" for an overlay pattern modeled on Messenger. A draggable chat head docks itself into the nearest screen edge, peeking seventy percent and hiding thirty; tapping it opens the guidance chat card, which scales out of the docked corner. An alternative Edge Panel style - a half-tucked pill handle resting fifty percent off-screen - opens the same chat card, and the style choice persists in Settings. One transcript backs every entry point, including Home's ask button, and an unread badge is plumbed for the day the engine becomes asynchronous. The engine itself is deterministic and rule-based, with suggested topics, tappable follow-up prompts, and an honest fallback when no rule matches.

4.5 The Student Community

The community module is a real feed on invoker-safe database views. Students post through a familiar "What's on your mind?" composer with real photo attachments from the system photo picker, uploaded to a private, per-user storage bucket. A post-as-anonymous switch strips author identity server-side - the author is nulled in the view, not merely hidden in the UI - and the choice is immutable once posted. Threads support likes, nested replies up to a database-enforced depth, and content reports that file into the moderation queue with reason and details. Posting rate limits and image-count limits are enforced in the database itself.

4.6 Profile and Settings

Profile is a pure identity page - avatar mark, name, program affiliation, signed-in-as, and version note. Settings is the operations hub, presented as one flat, header-free list of cards: the profile entry, the floating-assistant style switch, a change-password button that opens a modal bottom-sheet form, a change-school-details button that opens the same registry-backed editor used at sign-up, the Google Classroom connection card with its live sync-state note and disconnect action, the notifications preference row, and - after every operation - sign out. Guests see the subset that makes sense for a demo session and an exit-demo-mode row instead.

4.7 The teacher web dashboard

The teacher experience is a Next.js web application on the same Supabase project, gated by role. Section filters come from real enrollments; upcoming activities carry status labels; a day-and-week workload summary is computed from a database view rather than client-side guesswork. The proposed-deadline preview compares two candidate dates for a section and recommends the lighter one in plain language, with an honest caveat that it cannot see work outside the system. The report-triage view closes the loop opened by student reports: confirming a report creates a teacher-confirmed activity visible to the section and links the evidence to it; rejecting dismisses it. A coverage card states plainly that Google Classroom sync is not yet enabled.

4.8 Google Classroom readiness

The one major integration deliberately left switched off is live Google Classroom sync - and even that is staged rather than missing. The schema already carries activity sources with a unique source key of the form google_classroom:courseId:courseWorkId and a partial unique index, so repeated syncs converge instead of duplicating. An atomic, idempotent import RPC exists and is service-role only, proven by dedicated identity tests. A google_connections registry tracks connection state and last-sync without ever storing tokens, and both the Android connection card and the dashboard coverage card already render its state. Two demo activities carry Google Classroom source rows so the pipeline is visible today. Turning it on requires one Edge Function to perform the OAuth exchange and call the import RPC per coursework item - no app or schema changes.

Module	What it does	Status
Authentication	Email, Google, and guest sign-in; campus-first school-details sign-up	Live
Home dashboard	Date-grouped deadlines, due-soon counts, honest states	Live
Activity Beacon	Unified activity list; manual create, edit, delete	Live
Activity reports	Face-to-face evidence filed as pending for teacher triage	Live
Guidance Assistant	Floating chat head / edge panel; rule-based engine	Live
Community	Feed, photos, anonymous posting, likes, threads, reports	Live
Profile and Settings	Identity page; password, school details, assistant style	Live
Teacher dashboard	Section workload, deadline preview, report triage	Live
Google Classroom sync	Schema, RPC, connection state, demo evidence	Staged
Offline cache	Room cache and outbox design document	Designed
Notifications	Settings row present; delivery not yet implemented	Planned

Table 1. Feature inventory and current status.

5. Architecture and Technology

The system is deliberately small in moving parts and strict in boundaries. The Android client is Kotlin 2.2 on Jetpack Compose with Material 3, organized as screens plus ViewModels over repository interfaces, with a hand-rolled AppContainer providing dependency injection instead of a framework. Every network-adjacent capability - authentication, data access, storage - sits behind those repository interfaces, which is what allowed the project to start on fake demo repositories and swap in Supabase-backed implementations without touching a single screen.

The backend is a hosted Supabase project: PostgreSQL with row-level security on every table, Supabase Auth for email and Google credentials, PostgREST reached through the supabase-kt multiplatform client (version 3.6.0, pinned as the newest release compatible with the project's Kotlin toolchain), and a private storage bucket for community images. The database is defined by eleven versioned migrations - profiles and roles, academic structure, activities and sources, community, moderation, storage policies, and later the school-details registry, feed views, teacher workload view, sync RPC, and demo seed - all applied to the hosted project and reproducible from a clean start.

The teacher dashboard is a Next.js and TypeScript application deployed from the operator workspace, authenticating against the same Supabase project with teacher-role row-level security, so it can never read student data it is not entitled to see. The visual identity of the Android app - a library green on warm paper palette, the DM Sans variable typeface, content-shaped skeletons instead of spinners, and flat dense rows - was applied through two dedicated interface-precision passes and held constant since.

Layer	Technology	Notes
Android client	Kotlin 2.2, Jetpack Compose, Material 3	MVVM, repository interfaces, manual DI
Navigation	navigation-compose 2.9	Four bottom tabs plus pushed pages
Backend	Supabase (PostgreSQL, Auth, Storage)	Row-level security on every table
Data access	supabase-kt 3.6.0, postgrest-kt	Pinned for Kotlin compatibility
Web dashboard	Next.js, TypeScript	Same backend, teacher-role RLS
Database	11 versioned SQL migrations	Applied live; reproducible
Typeface	DM Sans variable font	SIL OFL licensed
Quality	117 unit tests; lint clean	Plus on-device end-to-end verification

Table 2. Technology stack summary.

6. Security and Privacy by Design

Security in StudentCodex is a property of the database first and the interface second. Every table is governed by row-level security policies that were written as tests before they were trusted: twenty-four policy scenarios covering roughly ninety assertions, executed against a real PostgreSQL 18.4 instance with a Supabase platform mock in the development sandbox, then confirmed live on the hosted project where anonymous requests return zero rows across every probed table. The policy test suite ships with the repository and runs against any psql, so the guarantees are re-checkable by anyone.

The client carries no secrets. Configuration flows from a git-ignored .env file into build-time-validated BuildConfig fields, and the Gradle build fails fast on a missing file, a malformed key, or any hint of service-role material - only the publishable key is ever packaged, and byte-level scans of every compiled dex confirm that no privileged key material ships in the APK. Google Classroom connection state is tracked in a registry that deliberately never stores tokens; when live sync arrives, refresh tokens will live only inside the Edge Function’s private store.

Privacy follows the same pattern of enforcement over convention. Anonymous community posts are anonymous at the view layer - author fields are nulled server-side before the API can see them, making the anonymity structural rather than cosmetic. Community images are stored in a private bucket with per-user folder policies. Guard triggers in the database cap reply depth, image counts, and posting rates, and a moderation queue receives content reports with reason and detail. Password changes go through Supabase Auth, and sessions persist through the platform’s encrypted session manager rather than custom storage.

7. Current Status and Known Limitations

As of October 24, 2026, every build-plan phase is complete. The hosted Supabase project carries all eleven migrations and serves live data; the Android application authenticates real accounts, reads and writes real activities, community posts, and profile data; the teacher dashboard is live with section workload and triage. The release candidate APK builds at roughly twenty-seven megabytes with one hundred seventeen unit tests green, zero lint errors, and every user-facing flow verified on an emulator against the hosted backend.

The project also states its limitations plainly, because a tool that wants to be trusted cannot hide them. Live Google Classroom sync is staged but not connected - the schema, import RPC, connection registry, and UI are ready, and activation is one Edge Function away. The app currently requires network for data: the offline cache and outbox described in the design document (Room cache, outbox, WorkManager retry-safe sync) is designed but not yet implemented, though authentication sessions do persist offline. The community composer attaches one photo per post from the app even though the schema permits

three. Notification preferences are exposed in Settings but delivery is not yet implemented. Each of these has a place in the roadmap recorded in the Development Chronicle.

8. Conclusion and Outlook

StudentCodex set out to replace a student's scattered academic scrolls with one bound volume - and the binding holds. One workload list with honest provenance, one floating assistant that admits what it does not know, one moderated community with structural anonymity, and one dashboard that lets teachers schedule with their eyes open: all on a single backend where row-level security is tested rather than assumed. The system is a release candidate in the strict sense - feature-complete for its stated scope, verified end to end, and honest about what remains.

The road ahead is documented, not improvised. The companion volume, the StudentCodex Development Chronicle and Roadmap, records how this system was built - every phase, fix, and refinement - and lays out the next steps: live Google Classroom synchronization, notifications and reminders, the offline cache, and the longer arc of workload analytics, a smarter assistant behind the same surface, and eventual multi-school expansion on the registry backbone that already exists.